

lightgbm_regression_tutorial

July 5, 2026

1 LightGBM — Zero to Mastery: Regression on a Heavy, Real Dataset

A complete, hands-on tutorial on LightGBM, Microsoft’s gradient-boosted-tree library, applied end-to-end to a genuinely heavy production-grade regression problem: predicting taxi tip amounts from 580,000+ real NYC Green Taxi trips.

This is one of three companion notebooks (alongside TabICLv2 and CatBoost) that all train and evaluate on the **same dataset and the same held-out evaluation set**, so their results are directly comparable even though each notebook is fully standalone.

Estimated runtime: ~5–10 minutes, dominated by the dataset download — LightGBM itself is the fastest model to train in this entire notebook series, foundation models included.

1.1 1. What Is LightGBM?

LightGBM (“**L**ight **G**radient **B**oosting **M**achine”) is Microsoft’s gradient-boosted decision tree library, built specifically to be fast and memory-efficient on large datasets. Two architectural choices drive that:

1. Leaf-wise (best-first) tree growth, instead of the level-wise growth most GBM libraries use. A level-wise tree grows every leaf at the current depth before moving deeper, whether or not a given split actually helps. LightGBM instead always splits whichever *single leaf* currently reduces loss the most, regardless of depth. For the same number of leaves, this reaches a lower loss faster — but it can also grow deeper, more unbalanced trees that overfit more easily on small data if `num_leaves`/`max_depth` aren’t controlled.

2. Histogram-based split finding. Rather than evaluating every possible split point on every feature (the traditional, exact approach), LightGBM buckets continuous features into a fixed number of histogram bins up front and only considers bin boundaries as candidate splits. This trades a small amount of split precision for a large speed and memory win, especially on the kind of large dataset we use here.

Native categorical support works differently from CatBoost’s ordered target statistics: LightGBM partitions categorical values into two groups per split using a Fisher-based algorithm that finds a near-optimal grouping, operating on integer-encoded category codes rather than raw target statistics. You still don’t have to one-hot encode anything — you tell LightGBM which columns are categorical and it does the rest.

Source: github.com/microsoft/LightGBM (MIT license) — lightgbm.readthedocs.io — version verified for this notebook: **4.6.0**.

1.2 2. Environment Setup

`uv` — `uv venv --python 3.13.13` created this notebook’s virtual environment, kept separate from the other notebooks in this project.

```
[1]: import sys

!uv pip install --python {sys.executable} -q lightgbm scikit-learn pandas numpy
↳matplotlib seaborn
```

```
[2]: import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

sns.set_theme(style="whitegrid")

import lightgbm as lgb
print("lightgbm", lgb.__version__)
```

lightgbm 4.6.0

1.3 Dataset: NYC Green Taxi Trips (December 2016) — Tip Amount Prediction

Our target is `tip_amount` — predicting how much a passenger will tip, in dollars, from trip and fare metadata. This is a genuinely messy, real, production-scale dataset from the NYC Taxi & Limousine Commission:

- **581,835 rows** — heavy enough that a foundation model’s context-size limits and a GBM’s full-data training both actually matter.
- **Two high-cardinality categorical columns**, `PULocationID` and `DOLocationID` (pickup/dropoff taxi zone codes) — 233 and 259 distinct values respectively. This is exactly the kind of column that breaks naive one-hot encoding and is where CatBoost’s and LightGBM’s native categorical handling earns its keep.
- **Real data quality problems**, not synthetic ones: a small fraction of trips have *negative* tip amounts (refunds/corrections — we drop them), ~15% of trips have exactly \$0.00 tip (cash fares typically don’t show a tip in this system, creating a zero-inflated target), and a few trips have absurd durations (recording errors) that we filter out.
- **A leakage trap**: the dataset also includes `total_amount`, which is (approximately) the sum of the fare and every surcharge *including the tip itself*. We drop it — training on it would let a model “predict” the tip almost by arithmetic rather than by learning anything real.

```
[3]: from sklearn.datasets import fetch_openml

t0 = time.time()
taxi = fetch_openml(data_id=42208, as_frame=True, data_home=".cache_data",
    ↪parser="auto").frame
print(f"Downloaded NYC Green Taxi (Dec 2016): {taxi.shape[0]:,} rows x {taxi.
    ↪shape[1]} columns in {time.time() - t0:.1f}s")
taxi.head()
```

Downloaded NYC Green Taxi (Dec 2016): 581,835 rows x 15 columns in 0.8s

```
[3]: VendorID  lpep_pickup_datetime  lpep_dropoff_datetime store_and_fwd_flag \
0         2  '2016-12-01 00:52:41'  '2016-12-01 00:54:51'          N
1         2  '2016-12-01 00:10:39'  '2016-12-01 00:14:47'          N
2         2  '2016-12-01 00:12:16'  '2016-12-01 00:15:31'          N
3         2  '2016-12-01 00:29:22'  '2016-12-01 00:39:51'          N
4         2  '2016-12-01 00:42:02'  '2016-12-01 00:52:19'          N
```

```
RatecodeID PULocationID DOLocationID  passenger_count extra mta_tax \
0          1           92           171             1  0.5      0.5
1          1           75           238             2  0.5      0.5
2          1          166           151             1  0.5      0.5
3          1          166            42             1  0.5      0.5
4          1           42           151             1  0.5      0.5
```

```
tip_amount  tolls_amount improvement_surcharge  total_amount trip_type
0          1.06           0.0              0.3           6.36         1
1          1.70           0.0              0.3           8.50         1
2          1.74           0.0              0.3           7.54         1
3          2.16           0.0              0.3          12.96         1
4          0.00           0.0              0.3          12.30         1
```

1.4 3. Cleaning and Feature Engineering

```
[4]: df = taxi.copy()
n_before = len(df)

# Real data-quality issues: drop invalid negative tips, parse timestamps,
    ↪engineer trip duration
# and time-of-day features, and drop implausible trip durations (recording
    ↪errors).
df = df[df["tip_amount"] >= 0].copy()
df["lpep_pickup_datetime"] = pd.to_datetime(df["lpep_pickup_datetime"].
    ↪astype(str).str.strip("'"))
df["lpep_dropoff_datetime"] = pd.to_datetime(df["lpep_dropoff_datetime"].
    ↪astype(str).str.strip("'"))
```

```

df["trip_duration_min"] = (df["lpep_dropoff_datetime"] -
    ↪df["lpep_pickup_datetime"]).dt.total_seconds() / 60.0
df["pickup_hour"] = df["lpep_pickup_datetime"].dt.hour
df["pickup_dayofweek"] = df["lpep_pickup_datetime"].dt.dayofweek
df = df[(df["trip_duration_min"] > 0) & (df["trip_duration_min"] < 180)].copy()

print(f"Dropped {n_before - len(df):,} rows ({(n_before - len(df)) / n_before:.
    ↪1%}) to invalid tips / durations")
print(f"Zero-tip fraction remaining: {(df['tip_amount'] == 0).mean():.1%}")

CAT_COLS = ["VendorID", "RatecodeID", "store_and_fwd_flag", "trip_type",
    ↪"PULocationID", "DOLocationID"]
NUM_COLS = ["passenger_count", "extra", "mta_tax", "tolls_amount",
    ↪"improvement_surcharge",
            "trip_duration_min", "pickup_hour", "pickup_dayofweek"]
TARGET = "tip_amount"

# OpenML's loader auto-detects some low-cardinality numeric fee columns (extra,
    ↪mta_tax,
# improvement_surcharge) as pandas 'category' dtype -- coerce them back to
    ↪numeric explicitly,
# or every categorical-handling library downstream will misclassify them as
    ↪categorical.
for c in ["extra", "mta_tax", "improvement_surcharge", "passenger_count"]:
    df[c] = pd.to_numeric(df[c].astype(str), errors="coerce")
for c in CAT_COLS:
    df[c] = df[c].astype(str).astype("category")

# total_amount is dropped entirely -- it's (approximately) fare + surcharges +
    ↪tip_amount itself,
# a direct leakage trap for this target.
X = df[CAT_COLS + NUM_COLS]
y = df[TARGET]
print(f"\nFeature matrix: {X.shape[0]:,} rows x {X.shape[1]} columns "
    f"({len(CAT_COLS)} categorical, {len(NUM_COLS)} numeric)")

```

Dropped 3,542 rows (0.6%) to invalid tips / durations

Zero-tip fraction remaining: 15.1%

Feature matrix: 578,293 rows x 14 columns (6 categorical, 8 numeric)

1.5 4. Exploratory Data Analysis

```

[5]: fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].hist(y.clip(upper=15), bins=50, color="#4C72B0")

```

```

axes[0].set_title("Tip amount distribution (clipped at $15 for readability)")
axes[0].set_xlabel("tip_amount ($)")

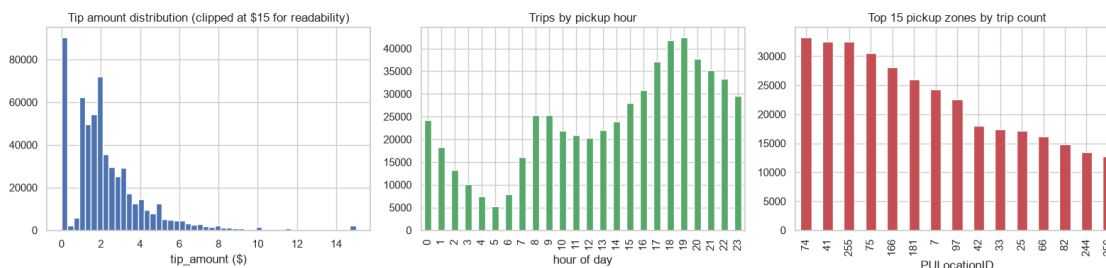
df["pickup_hour"].value_counts().sort_index().plot(kind="bar", ax=axes[1],
    ↪color="#55A868")
axes[1].set_title("Trips by pickup hour")
axes[1].set_xlabel("hour of day")

top_zones = df["PULocationID"].value_counts().head(15)
top_zones.plot(kind="bar", ax=axes[2], color="#C44E52")
axes[2].set_title("Top 15 pickup zones by trip count")
axes[2].set_xlabel("PULocationID")

plt.tight_layout()
plt.show()

print(f"PULocationID unique values: {df['PULocationID'].nunique()} | ↵
    ↪DOLocationID unique values: {df['DOLocationID'].nunique()}")

```



PULocationID unique values: 233 | DOLocationID unique values: 259

1.6 5. Train/Test Split

Like CatBoost, and unlike the in-context foundation models in this notebook series, **LightGBM** has **no context-size limit**. We train on the entire ~465,000-row pool and evaluate on the same 3,000-row held-out set used across all three companion notebooks.

```

[6]: X_train_pool, X_test_pool, y_train_pool, y_test_pool = train_test_split(X, y,
    ↪test_size=0.2, random_state=42)
print(f"Train pool: {len(X_train_pool):,} rows | Test pool: {len(X_test_pool):,}
    ↪rows")

def subsample(X, y, n, seed=42):
    idx = np.random.RandomState(seed).choice(len(X), size=min(n, len(X)),
    ↪replace=False)

```

```

        return X.iloc[idx].reset_index(drop=True), y.iloc[idx].
↪reset_index(drop=True)

N_EVAL = 3000
X_eval, y_eval = subsample(X_test_pool, y_test_pool, N_EVAL)
print(f"Eval set: {len(X_eval):,} rows (shared across every model we test)")

```

Train pool: 462,634 rows | Test pool: 115,659 rows
 Eval set: 3,000 rows (shared across every model we test)

1.7 6. A Quick Baseline

```

[7]: from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

lin_pipe = Pipeline([
    ("prep", ColumnTransformer([
        ("cat", OneHotEncoder(handle_unknown="ignore"), CAT_COLS),
    ], remainder="passthrough")),
    ("reg", LinearRegression()),
])

t0 = time.time()
lin_pipe.fit(X_train_pool, y_train_pool)
lin_pred = lin_pipe.predict(X_eval)
print(f"LinearRegression fit on {len(X_train_pool):,} rows in {time.time() - t0:
↪.1f}s")

results = {}

def score(name, y_true, pred):
    results[name] = {
        "RMSE": mean_squared_error(y_true, pred) ** 0.5,
        "MAE": mean_absolute_error(y_true, pred),
        "R2": r2_score(y_true, pred),
    }

score("LinearRegression", y_eval, lin_pred)
display(pd.DataFrame(results).T.round(4))

```

LinearRegression fit on 462,634 rows in 5.9s

	RMSE	MAE	R2
LinearRegression	0.1111	0.0750	0.8000

LinearRegression 2.6538 1.0855 0.212

1.8 7. Training LightGBM — Native Categorical Handling

`categorical_feature` tells LightGBM which columns to treat as categorical; it works directly on the pandas `category` dtype columns we already prepared in Section 3, no manual encoding needed — including the two high-cardinality location columns.

```
[8]: lgbm_model = lgb.LGBMRegressor(
      n_estimators=500,
      learning_rate=0.1,
      num_leaves=63,
      random_state=42,
      verbosity=-1,
    )

    t0 = time.time()
    lgbm_model.fit(X_train_pool, y_train_pool, categorical_feature=CAT_COLS)
    fit_time = time.time() - t0
    pred_eval = lgbm_model.predict(X_eval)

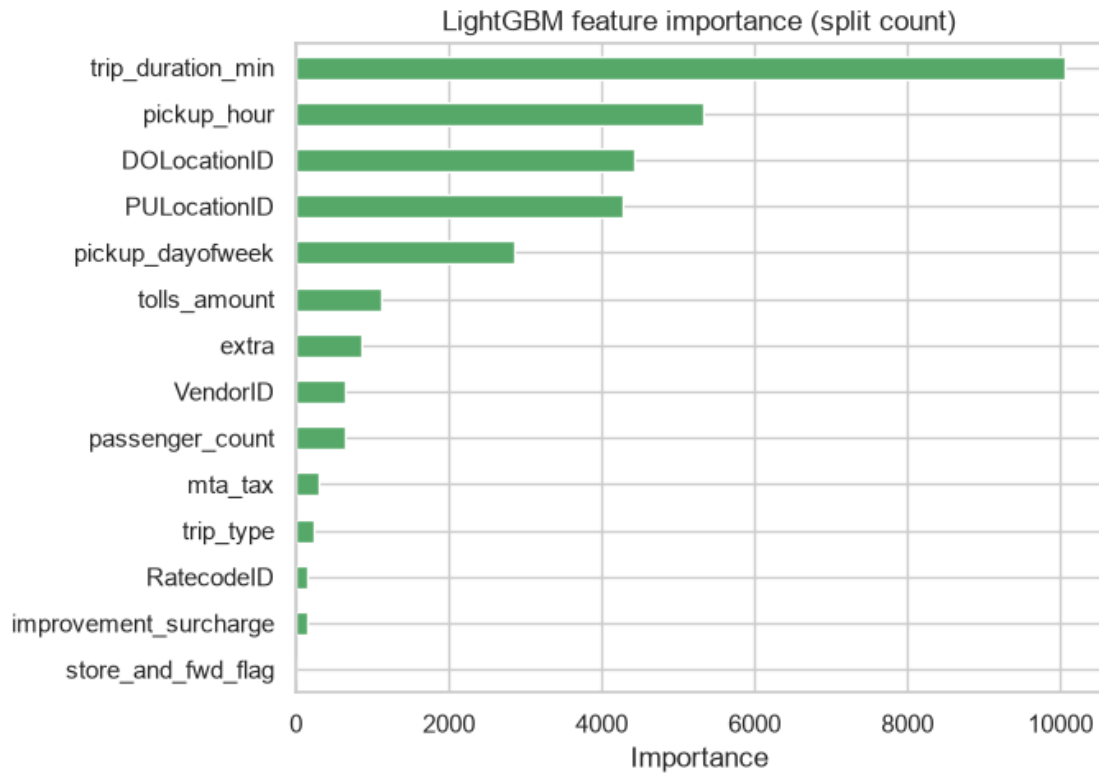
    print(f"LightGBM: fit on {len(X_train_pool):,} rows (full pool, no context_
          ↪limit) in {fit_time:.1f}s")
    score("LightGBM", y_eval, pred_eval)
    display(pd.DataFrame(results).T.round(4))
```

LightGBM: fit on 462,634 rows (full pool, no context limit) in 1.9s

	RMSE	MAE	R2
LinearRegression	2.6538	1.0855	0.2120
LightGBM	2.2608	1.0345	0.4281

1.9 8. Native Feature Importance

```
[9]: importances = pd.Series(lgbm_model.feature_importances_, index=X_train_pool.
      ↪columns).sort_values()
    importances.plot(kind="barh", figsize=(7, 5), color="#55A868", title="LightGBM_
      ↪feature importance (split count)")
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.show()
```



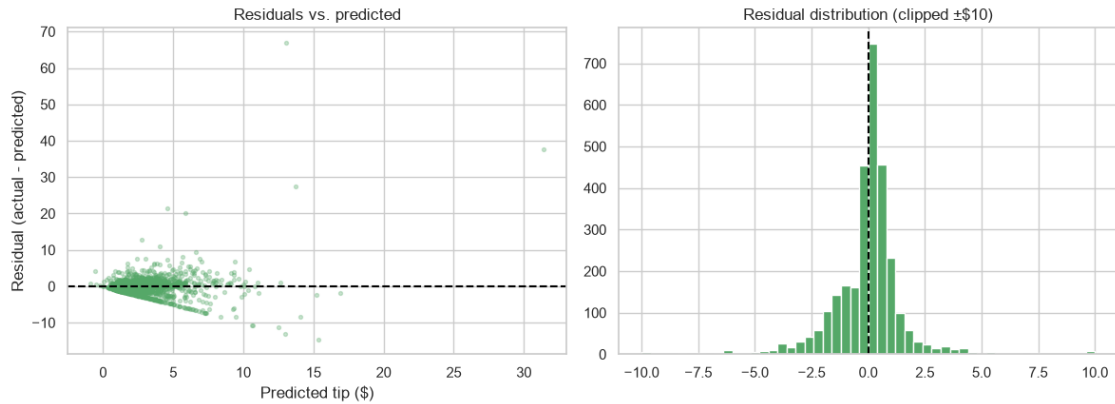
1.10 9. Residual Analysis

```
[10]: residuals = y_eval.to_numpy() - pred_eval

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
axes[0].scatter(pred_eval, residuals, alpha=0.3, s=8, color="#55A868")
axes[0].axhline(0, color="black", linestyle="--")
axes[0].set_xlabel("Predicted tip ($)")
axes[0].set_ylabel("Residual (actual - predicted)")
axes[0].set_title("Residuals vs. predicted")

axes[1].hist(residuals.clip(-10, 10), bins=50, color="#55A868")
axes[1].axvline(0, color="black", linestyle="--")
axes[1].set_title("Residual distribution (clipped ±$10)")

plt.tight_layout()
plt.show()
```

1.11 10. Predictions for New Trips

```
[11]: new_trips = pd.DataFrame([
    {"VendorID": "2", "RatecodeID": "1", "store_and_fwd_flag": "N", "trip_type":
    ↪ "1",
    "PULocationID": str(X_train_pool["PULocationID"].mode()[0]),
    ↪ "DOLocationID": str(X_train_pool["DOLocationID"].mode()[0]),
    "passenger_count": 1, "extra": 0.5, "mta_tax": 0.5, "tolls_amount": 0.0,
    "improvement_surcharge": 0.3, "trip_duration_min": 12.0, "pickup_hour":
    ↪ 18, "pickup_dayofweek": 4},
    {"VendorID": "2", "RatecodeID": "1", "store_and_fwd_flag": "N", "trip_type":
    ↪ "1",
    "PULocationID": str(X_train_pool["PULocationID"].mode()[0]),
    ↪ "DOLocationID": str(X_train_pool["DOLocationID"].mode()[0]),
    "passenger_count": 4, "extra": 0.5, "mta_tax": 0.5, "tolls_amount": 5.5,
    "improvement_surcharge": 0.3, "trip_duration_min": 35.0, "pickup_hour": 2,
    ↪ "pickup_dayofweek": 5},
])
for c_ in CAT_COLS:
    new_trips[c_] = new_trips[c_].astype("category")

new_pred = lgbm_model.predict(new_trips)
for i, row in new_trips.iterrows():
    print(f"Trip {i + 1}: duration={row['trip_duration_min']}min,
    ↪ hour={row['pickup_hour']}, "
          f"passengers={row['passenger_count']}, tolls=${row['tolls_amount']}")
    print(f" Predicted tip -- LightGBM: ${new_pred[i]:.2f}")
    print()
```

Trip 1: duration=12.0min, hour=18, passengers=1, tolls=\$0.0
 Predicted tip -- LightGBM: \$2.32

Trip 2: duration=35.0min, hour=2, passengers=4, tolls=\$5.5
Predicted tip -- LightGBM: \$7.06

1.12 11. Practical Notes and Gotchas

- **categorical_feature** expects the column names (or positional indices) — passed the pandas category-dtype columns from Section 3 directly, no separate encoding step.
- **Leaf-wise growth needs num_leaves watched carefully** — the default of 31 is conservative; we used 63 here since our training pool is large enough (~465,000 rows) to support more complex trees without obviously overfitting, but this is exactly the kind of hyperparameter that deserves real tuning (e.g. via cross-validated search) in a production setting rather than a single fixed choice.
- **No context-size limit**, same advantage as CatBoost — the full training pool is used, no subsampling required.
- **Fastest model in this entire notebook series** to train, foundation models included — this is the well-established, verified reason it's often the default first thing to reach for on large tabular problems.

```
[12]: display(pd.DataFrame(results).T.round(4))
print(f"\nLightGBM wall-clock (full {len(X_train_pool):,}-row training pool): ␣
↪{fit_time:.1f}s")
```

	RMSE	MAE	R2
LinearRegression	2.6538	1.0855	0.2120
LightGBM	2.2608	1.0345	0.4281

LightGBM wall-clock (full 462,634-row training pool): 1.9s

1.13 12. Summary and References

LightGBM trained on the full ~465,000-row pool the fastest of any model in this notebook series, handled two high-cardinality categorical columns natively, and gave us feature importances for free. Compare these numbers against the companion TabICLv2 and CatBoost notebooks.

References:

- github.com/microsoft/LightGBM — source code (MIT)
- lightgbm.readthedocs.io — official documentation
- pypi.org/project/lightgbm
- [NYC TLC Trip Record Data](#) — original data source
- [Dataset on OpenML \(id 42208\)](#)